Agentic AI

Complete Developer Study Notes

Concepts • Practical Examples • Projects • Interview Preparation

Prepared by: Rahul Yadav

20 Pages

Rahul Yadav

1 Introduction to Agentic AI

1.1 What is Agentic AI ?

- Agentic AI refers to AI systems that can plan, reason, take actions and achieve goals autonomously with minimal human intervention.
- Unlike traditional AI which only responds to queries, Agentic AI can understand the goal, make decisions, use tools and complete complex multi-step tasks.
- It combines the capabilities of LLMs with planning, memory, tool use and reasoning to work like an intelligent agent.

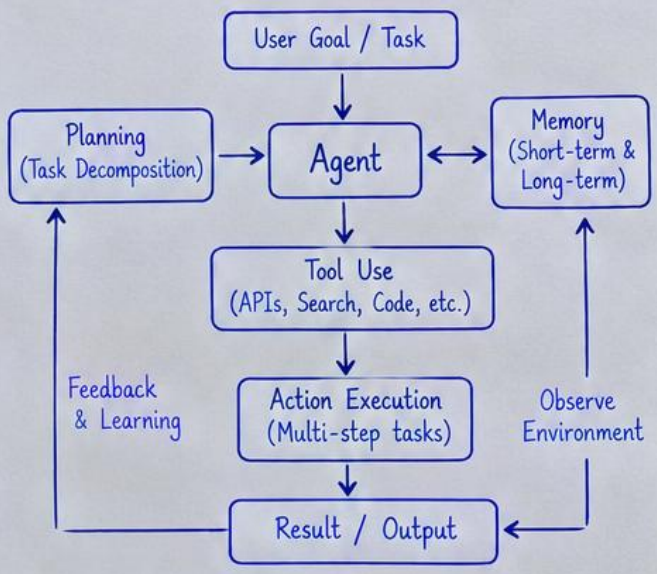
1.2 Key Characteristics

1. Autonomy : Can operate with minimal human input.
2. Goal-Oriented : Works towards achieving specific goals.
3. Planning & Reasoning : Breaks complex tasks into smaller steps.
4. Tool Use : Can use external tools, APIs and systems.
5. Memory : Maintains short-term and long-term memory.
6. Adaptability : Learns from feedback and environment.
7. Multi-step Execution : Can handle complex workflows.
8. Self-Improvement : Can improve based on experience.

1.3 Agentic AI vs Traditional AI

Aspect	Traditional AI	Agentic AI
Nature	Reactive	Proactive and Autonomous
Goal Handling	Single step	Multi-step and complex
Decision Making	Predefined rules / model response	Plans and reasons dynamically
Tool Usage	Usually not	Can use external tools
Memory	Limited / None	Short-term and long-term
Adaptability	Fixed behavior	Learns and adapts
Example	ChatGPT (Q&A)	AI Agent for research, analysis, automation

1.4 How Agentic AI Works ?



1.5 Examples of Agentic AI

1. Research Agent : Automatically search, read, summarize and create reports on a topic.
2. Code Agent : Write, debug and execute code using tools like IDE, terminal and documentation.
3. Data Analyst Agent : Analyze data, generate insights and create visualizations.
4. Personal Assistant Agent : Manage emails, calendar, travel plans and daily tasks.
5. Web Automation Agent : Interact with websites to complete tasks (form filling, booking, etc).
6. Multi-Agent System : Multiple agents collaborating to solve complex problems.
7. Enterprise Agent : Automate business workflows using internal tools and APIs.

1.6 Real-World Applications

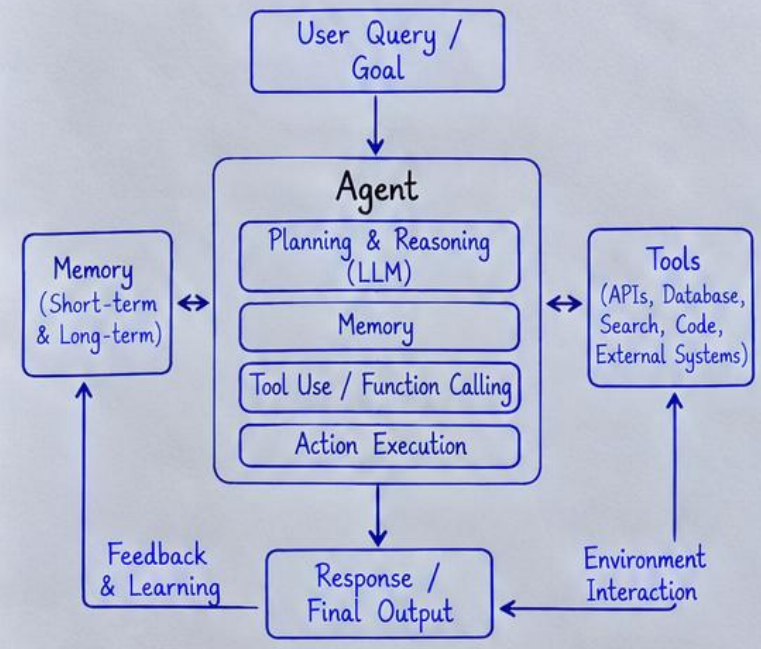
1. Customer Support : Intelligent agents to handle user queries and automate resolutions.
2. Software Development : Code generation, testing and deployment automation.
3. Healthcare : Patient data analysis and research assistance.
4. Finance : Market analysis, risk assessment and report generation.
5. Education : Personalized learning and tutoring agents.
6. Operations : Process automation and workflow orchestration.
7. Research : Literature review and scientific analysis.

Rahul Yadav

2 Agent Architecture and Components

2.1 High Level Architecture of an Agent

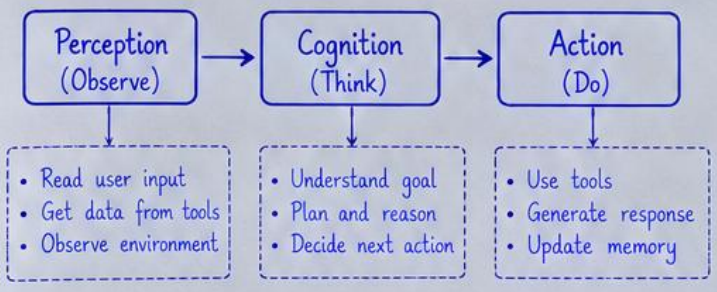
An AI agent consists of an LLM as the core reasoning engine, memory to store context, tools to interact with external systems and a planning mechanism to break down tasks and take actions.



2.2 Core Components

1. **LLM (Reasoning Engine)**
   - Understands the goal, plans steps and decides actions.
   - Examples: GPT-4, Claude, Llama, Gemini.
2. **Memory**
   - Stores conversation history, user proferences and domain knowledge.
   - Types: Short-term (in-context), Long-term (vector database), Episodic memory.
3. **Tools (Function Calling)**
   - Allows the agent to interact with external systems like APIs, databases, web, code execution, etc.
   - Examples: Search API, Weather API, Calculator, Database query, Email, etc.
4. **Planning & Reasoning**
   - Breaks down complex tasks into smaller steps.
   - Selects appropriate tools and decides the next action.
   - Techniques: CoT, ReAct, Plan-and-Execute, Reflection.
5. **Action Execution**
   - Executes the selected tool or generates a response.
   - Handles errors and retries if needed.
6. **Feedback & Learning**
   - Learns from past interactions and improves over time.
   - Uses techniques like self-reflection, reinforcement and human feedback.

2.3 Agent Perception, Cognition and Action



2.4 Types of Agent Architectures

Type	Description	Use Case
Single Agent	Single LLM with tools and memory	Simple tasks, personal assistant
Agent with Tools	LLM + external tools/APIs	Real-time data, code execution
Multi-Agent	Multiple agents collaborating	Complex workflows, large tasks
Hierarchical Agent	Manager agent with sub-agents	Enterprise automation

2.5 Example: Simple Agent Workflow

1. User provides a goal (e.g., "Find best laptops under 50000").
2. Agent understands the task and creates a plan.
3. Agent decides to use a web search tool.
4. Executes the search and gets results.
5. Analyzes the results and prepares a summary.
6. Returns the final response to the user.

2.6 Key Takeaways

- An agent is more than just an LLM; it can plan, reason, use tools and learn.
- Core components: LLM, memory, tools, planning and action execution.
- Different architectures are used based on the complexity of tasks.
- Agents enable automation of real-world, multi-step tasks.

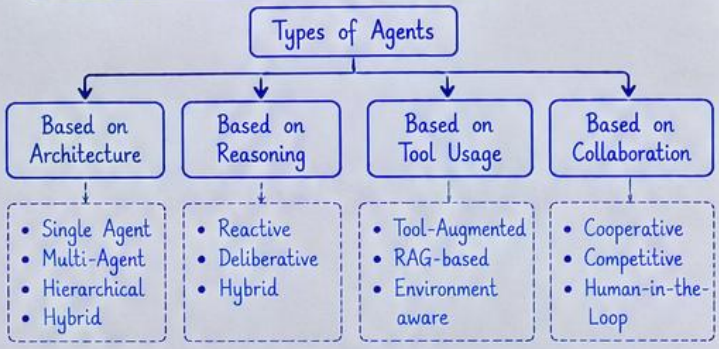
Rahul Yadav

3 Types of Agents

3.1 Overview

- Agentic AI has different types of agents based on their architecture, capabilities and use cases.
- Each type of agent is designed to handle specific tasks and complexity.
- In real-world applications, multiple types of agents are often combined together.

3.2 Classification of Agents



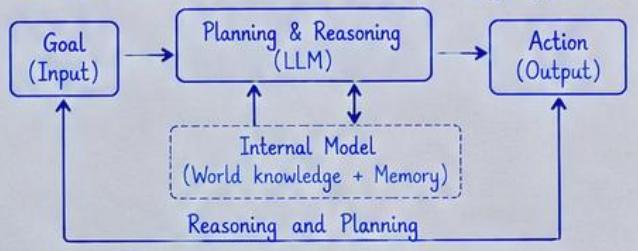
3.3 Reactive Agent

- Responds to the current environment without maintaining a long-term plan or memory.
- Uses simple rules or policies.
- Works well for simple and time-sensitive tasks.
- Example: Chatbot, real-time alert system.



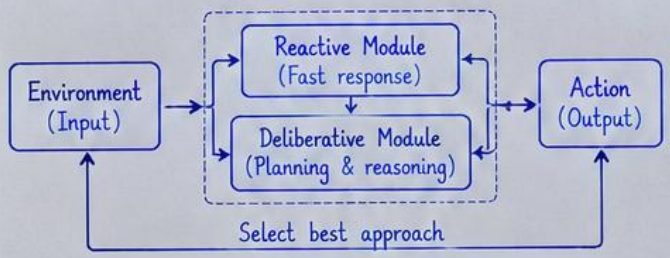
3.4 Deliberative Agent

- Plans and reasons before taking action.
- Uses internal models and long-term goals.
- Handles complex tasks by considering future outcomes.
- Example: Research assistant, task planning agent.



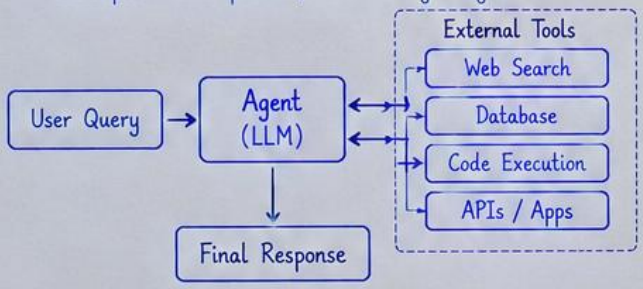
3.5 Hybrid Agent

- Combines reactive and deliberative approaches.
- Uses quick responses for simple tasks and planning for complex tasks.
- Example: Personal assistant, autonomous workflow agent.



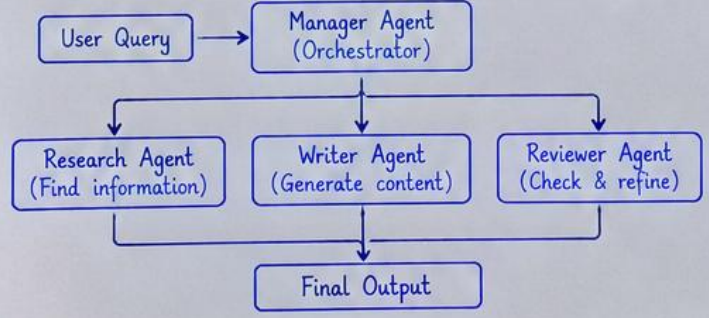
3.6 Tool-Augmented Agent

- Uses external tools, APIs and systems to perform tasks.
- Can search the web, query databases, run code, access files and use third-party services.
- Greatly extends the capabilities of LLMs.
- Example: Travel planner, data analyst agent.



3.7 Multi-Agent System

- Multiple agents work together to solve complex tasks.
- Each agent has a specific role (e.g., planner, researcher, executor, reviewer).
- Agents can communicate and collaborate.
- Example: Research workflow, content generation pipeline.



3.8 Hierarchical Agent

- Organized in a hierarchy with a manager agent and sub-agents.
- Manager agent breaks down the task and assigns it to sub-agents.
- Useful for large and complex workflows.
- Example: Enterprise automation, software development.



Rahul Yadav

4 Agent Frameworks and Tools

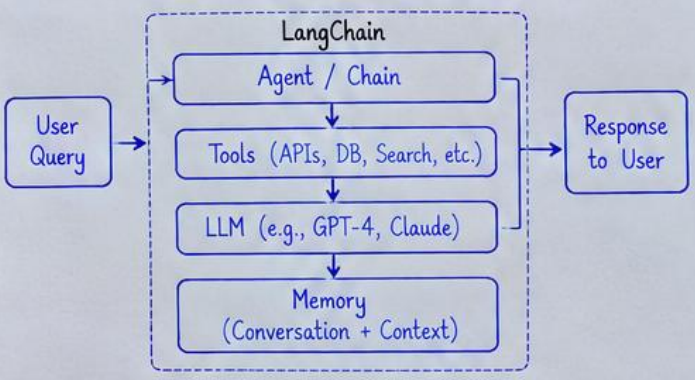
4.1 What are Agent Frameworks ?

- Agent frameworks are libraries and platforms that provide building blocks to create, configure, and run AI agents.
- They simplify development by handling common tasks like tool integration, memory management, planning and orchestration.
- Developers can focus on application logic instead of building everything from scratch.

4.2 Popular Agent Frameworks

Framework	Language	Key Features	Use Cases
LangChain	Python	Chains, Agents, Tools, Memory	RAG, Chatbots, Custom Agents
LangGraph	Python	Graph-based agents, Multi-agent workflows	Complex workflows, State management
AutoGen	Python	Multi-agent conversation	Collaborative agents, Automation
CrewAI	Python	Role-based agents, Easy configuration	Team of agents, Business automation
Semantic Kernel	Python/C#/JS	Enterprise ready, Plugins and tools	Enterprise apps, Tool integration
LlamaIndex	Python	Data ingestion, RAG and agents	Knowledge agents, Document Q&A

4.3 LangChain Architecture (Overview)

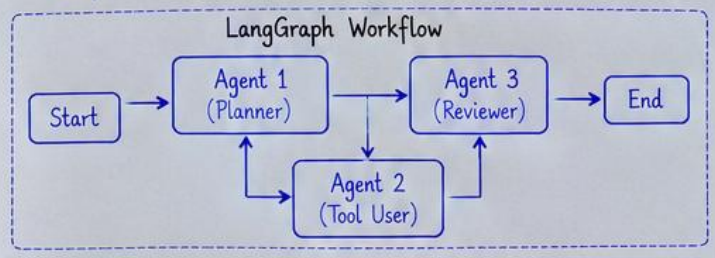


4.4 LangChain Components

1. LLM : Interface to interact with language models (e.g., OpenAI, Anthropic, local models).
2. Prompt Templates : Reusable templates for instructions.
3. Chains : Combine multiple steps together.
4. Agents : Use tools to decide and take actions.
5. Tools : External integrations (search, APIs, database, code).
6. Memory : Maintain conversation history and context.
7. Output Parsers : Structure the model output.

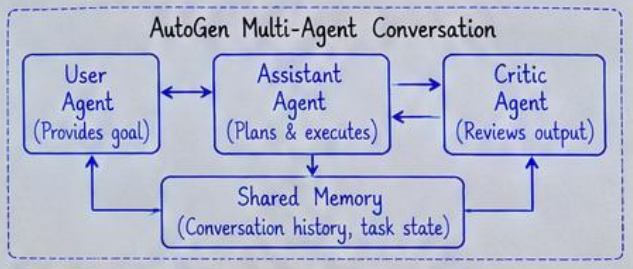
4.5 LangGraph (Graph based Agent)

- LangGraph allows building stateful, graph-based multi-agent workflows.
- It is useful for complex tasks where agents need to collaborate, loop or maintain state.
- Each node in the graph can represent an agent, tool or logic step.



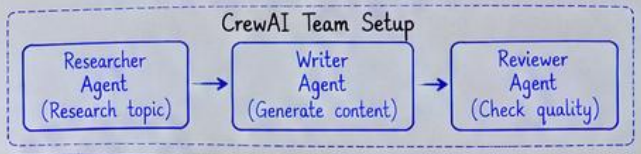
4.6 AutoGen (Multi-Agent Framework)

- Developed by Microsoft for building multi-agent conversational systems.
- Supports different types of agents (Assistant, User, Critic, etc.).
- Agents can communicate, share context and work together to solve tasks.



4.7 CrewAI (Role Based Agents)

- CrewAI makes it easy to create a team of agents with specific roles and goals.
- Each agent has a role, backstory and set of tools.
- Useful for real-world business workflows.



4.8 Choosing the Right Framework

Criteria	LangChain	LangGraph	AutoGen	CrewAI
Best for	General purpose	Complex workflows	Multi-agent conversations	Role-based teams
Ease of use	Easy	Moderate	Moderate	Easy
Multi-agent support	Yes	Yes (Advanced)	Yes	Yes
Tool integration	Wide support	Wide support	Good	Good
Ideal use case	RAG, chatbots	Stateful agents	Collaborative agents	Business workflows

4.9 Key Takeaways

1. Use frameworks to speed up agent development.
2. Choose the right framework based on your use case.
3. All frameworks support tool integration and memory.
4. For complex multi-agent workflows, LangGraph or AutoGen are better options.

Rahul Yadav

5 Planning and Reasoning in Agentic AI

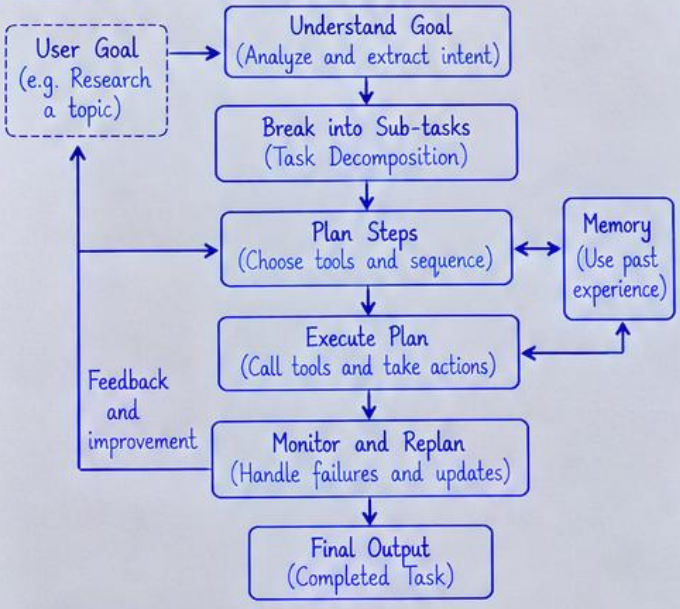
5.1 Why Planning and Reasoning?

- Planning and reasoning help an agent break down complex goals into smaller steps, make better decisions and achieve the desired outcome.
- Without planning, agents may take random actions, get stuck or fail to complete long tasks.
- Reasoning allows the agent to understand the environment, evaluate options and choose the best next step.

5.2 Types of Planning

1. Goal Planning : Plan a sequence of steps to achieve a specific goal.
2. Task Decomposition : Break complex tasks into smaller sub-tasks.
3. Hierarchical Planning : Plan at high level and then refine into low-level steps.
4. Sequential Planning : Execute tasks step by step.
5. Conditional Planning : Handle different scenarios based on conditions.
6. Replanning : Adjust the plan when the environment changes.

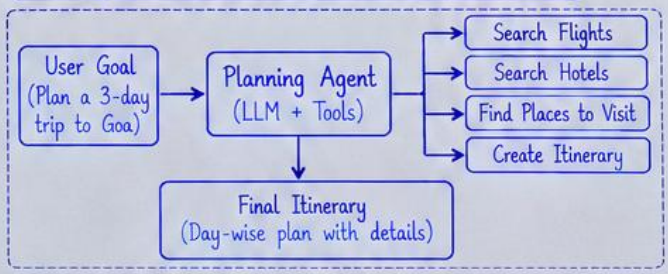
5.3 Planning Workflow



5.4 Common Planning Techniques

1. Chain of Thought (CoT) : Step-by-step reasoning to solve a problem.
2. Tree of Thoughts (ToT) : Explore multiple reasoning paths and choose the best one.
3. ReAct : Combine reasoning with action (thought → action → observation).
4. Plan-and-Execute : Create a full plan first and then execute each step.
5. Self-Reflection : Evaluate the result and improve the plan if needed.
6. Few-shot Planning : Learn planning patterns from examples.

5.5 Example: Travel Planning Agent



5.6 Reasoning Techniques

1. Deductive Reasoning : Apply logical rules to reach a conclusion.
2. Inductive Reasoning : Learn patterns from examples.
3. Abductive Reasoning : Find the most likely explanation.
4. Commonsense Reasoning : Use real-world knowledge.
5. Multi-Step Reasoning : Break down complex problems.
6. Tool-Augmented Reasoning : Use external tools to get information before reasoning.

5.7 Chain of Thought (CoT) Example

Question : If I have 3 flights per day and need to compare prices for 2 days, how many total searches are required?

Reasoning Steps :

1. Identify number of flights per day = 3
2. Number of days to check = 2
3. Total searches = $3 \times 2 = 6$

Answer : 6 searches are required.

5.8 ReAct Pattern (Reasoning + Action)

Thought 1	→	I need to find current weather in Goa.
Action 1	→	Search_Weather("Goa")
Observation 1	→	Weather is 30°C and sunny.
Thought 2	→	Now find best hotels under 5000.
Action 2	→	Search_Hotels("Goa", budget=5000)
Observation 2	→	List of hotels with prices.
Final Answer	→	Recommend top 3 hotels to the user.

5.9 Key Takeaways

- Planning helps agents handle complex, long-running tasks.
- Reasoning improves decision making and accuracy.
- Techniques like CoT, ReAct and Plan-and-Execute are widely used.
- Agents can replan based on feedback and changing environment.

Rahul Yadav

6 Tool Use and Function Calling in Agentic AI

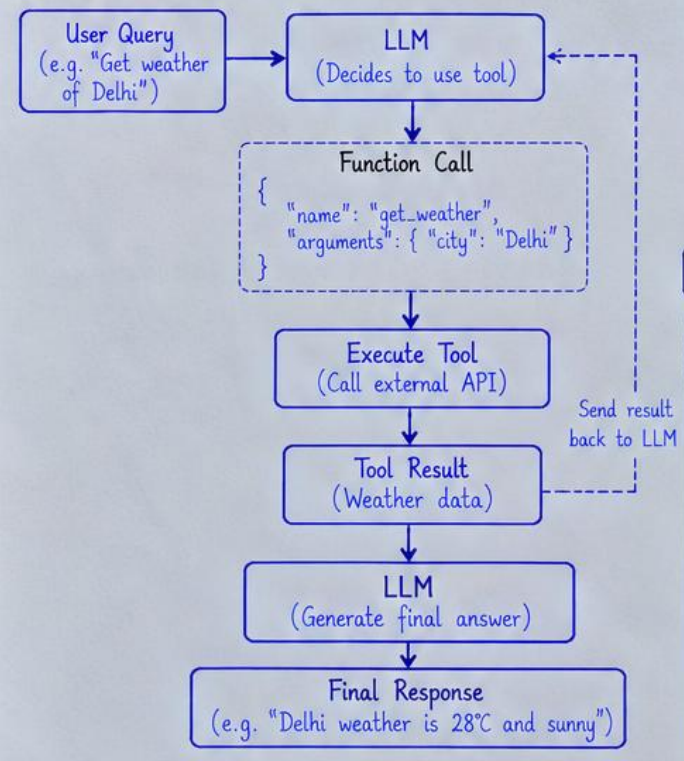
6.1 What is Tool Use ?

- Tool use allows an agent to interact with external systems, APIs, databases, websites or custom tools to perform real-world tasks.
- The LLM decides when to use a tool based on the user request and available tool descriptions.
- It extends the capabilities of agents beyond text generation to take real actions in the real world.

6.2 What is Function Calling ?

- Function calling is a mechanism where the LLM returns a structured output (JSON) to call a specific function or tool.
- Instead of generating plain text, the model generates a function name and arguments.
- The application executes the function, gets the result and sends it back to the LLM for final response.

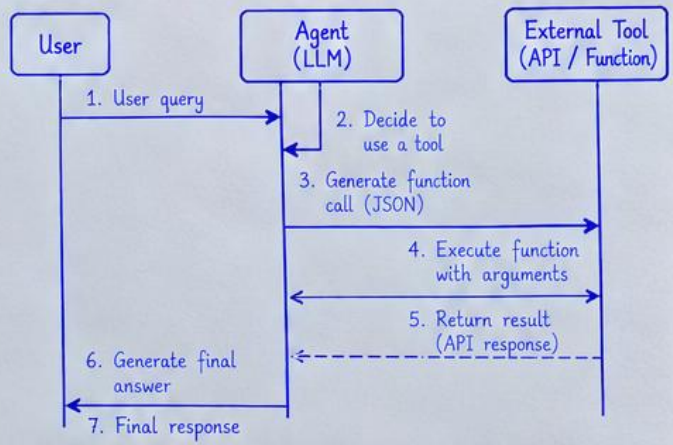
6.3 How Tool Use Works ?



6.4 Example Tools Agents Can Use

1. Web Search : Search latest information from internet.
2. Weather API : Get real-time weather information.
3. Database : Query internal company data.
4. Code Execution : Run Python code for calculations.
5. File System : Read, write and process files.
6. Custom APIs : Call business-specific APIs (e.g. flight booking).

6.5 Function Calling Flow (Detailed)



6.6 Function Schema (JSON Example)

```
{
  "name": "get_weather",
  "description": "Get current weather information for a city",
  "parameters": {
    "type": "object",
    "properties": {
      "city": {
        "type": "string",
        "description": "Name of the city (e.g. Delhi)"
      },
      "unit": {
        "type": "string",
        "enum": ["celsius", "fahrenheit"],
        "description": "Temperature unit"
      }
    },
    "required": ["city"]
  }
}
```

6.7 Python Example (OpenAI Function Calling)

```
from openai import OpenAI
client = OpenAI()
functions = [
    {
        "name": "get_weather",
        "description": "Get current weather information",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {"type": "string", "description": "City name"}
            },
            "required": ["city"]
        }
    }
]
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "What's the weather in Delhi?"}],
    functions=functions,
    function_call="auto"
)
print(response.choices[0].message.tool_calls)
```

6.8 Key Takeaways

- Tool use allows agents to interact with the real world.
- Function calling provides structured and reliable way to call tools.
- Agents can use multiple tools sequentially.
- Proper tool descriptions help the LLM decide correctly.
- Tool use is essential for building useful and production-ready agents.

Rahul Yadav

7 Memory in Agentic AI

7.1 What is Memory in Agentic AI ?

- Memory allows an agent to store, retrieve and use past information to make better decisions.
- It helps the agent maintain context across multiple interactions.
- Without memory, the agent behaves like a stateless system and forgets previous conversation and actions.
- Memory is essential for long-term goals, personalization and complex multi-step tasks.

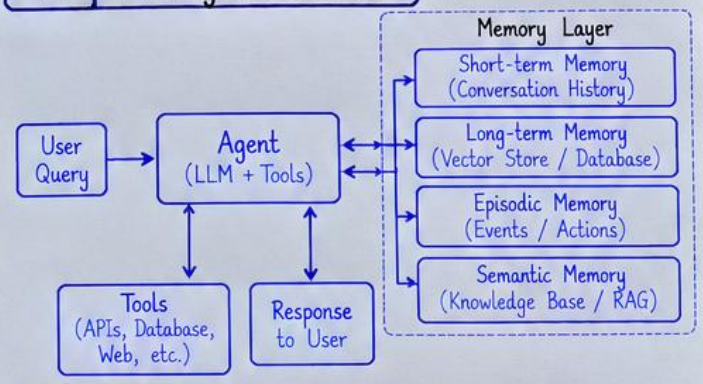
7.2 Why Memory is Important ?

- Helps in maintaining conversation context.
- Improves decision making with past knowledge.
- Allows personalization and user preferences.
- Useful for long-running tasks and autonomous agents.
- Reduces repetitive questions and improves efficiency.

7.3 Types of Memory

Type	Description	Examples	Use Cases
Short-term Memory	Stores recent conversation in current session.	Chat history, recent user input	Maintaining context in a single conversation
Long-term Memory	Stores important information for future use.	User preferences, domain knowledge	Personalization, continuous learning
Episodic Memory	Stores past events and experiences.	Previous tasks, actions taken, results	Learning from past interactions
Semantic Memory	Stores factual knowledge and world information.	Company info, general knowledge	Answering factual questions, RAG

7.4 Memory Architecture



7.5 Short-term Memory (Conversation Memory)

- Stores recent messages in the current session.
- Helps the agent understand the immediate context.
- Usually implemented as a sliding window of last N messages.
- Example: Keep last 5-10 user and assistant messages.

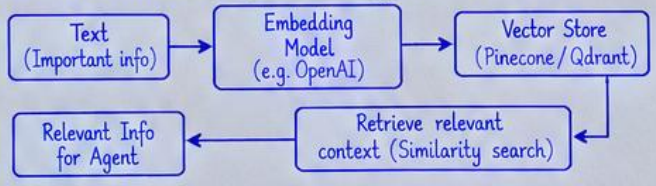
Conversation Memory (Last 5 messages)

- User: What is the weather in Goa?
- Assistant: Goa is sunny with 30°C.
- User: What about tomorrow?
- Assistant: Tomorrow it will be 28°C.
- User: Suggest some hotels there.

Kept in short-term memory

7.6 Long-term Memory (Vector Store)

- Stores important information for future use.
- Implemented using vector databases (e.g., Pinecone, Qdrant).
- Converts text to embeddings and stores in a vector store.
- Retrieves relevant information using semantic search.



7.7 Episodic Memory (Event Memory)

- Stores past actions, events and experiences.
- Helps agent learn from previous interactions.
- Useful for long-running tasks and autonomous agents.
- Example: Remembering that user booked a flight last week.

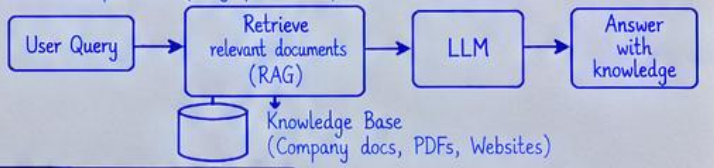
Episodic Memory Example

Event: Booked flight to Goa
Date: 12 Oct 2024
Details: From Delhi to Goa, 2 tickets

Event: Asked for hotel suggestion
Date: 12 Oct 2024
Details: Looking for beachside hotels

7.8 Semantic Memory (Knowledge Base)

- Stores general knowledge and domain-specific information.
- Often implemented using RAG (Retrieval-Augmented Generation).
- Retrieves relevant documents and provides accurate responses.
- Example: Company policies, product documentation.



7.9 Key Takeaways

1. Memory is essential for building intelligent and autonomous agents.
2. Different types of memory serve different purposes.
3. Short-term memory handles immediate context, long-term memory stores important knowledge, episodic memory stores past events and semantic memory stores factual knowledge.
4. Vector databases and RAG are commonly used for long-term memory.
5. Good memory design significantly improves agent performance.

Rahul Yadav

8 Multi-Agent Systems in Agentic AI

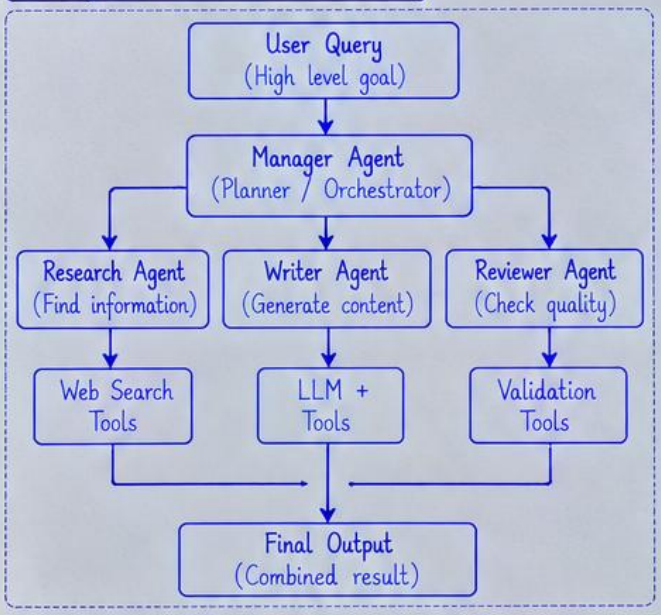
8.1 What are Multi-Agent Systems ?

- Multi-agent systems consist of multiple agents working together to solve complex tasks.
- Each agent has a specific role, goal and tools.
- Agents can collaborate, communicate and coordinate to achieve a common objective.
- Useful for large, real-world tasks where a single agent may not be sufficient.

8.2 Why Use Multi-Agent Systems ?

- Breaks complex tasks into smaller sub-tasks.
- Improves accuracy and reliability.
- Different agents can specialize in different skills.
- Enables parallel execution and better scalability.
- Useful in real-world applications like research, content creation, data analysis and enterprise workflows.

8.3 Common Architecture



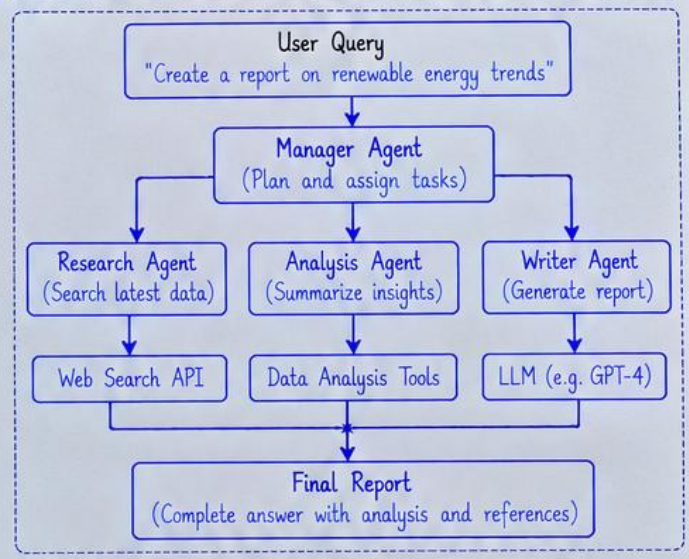
8.4 Types of Multi-Agent Collaboration

1. Hierarchical : One manager agent coordinates sub-agents.
2. Collaborative : Agents work together and share information.
3. Sequential : Agents work in a sequence (one after another).
4. Competitive : Multiple agents provide solutions and the best one is selected.
5. Hybrid : Combination of above approaches.

8.5 Communication Between Agents

- Agents can share messages, context and intermediate results.
- Communication can be done using:
  - Shared memory (database / vector store)
  - Message Passing (direct agent-to-agent)
  - Central Orchestrator (manager agent)

8.6 Example: Research Assistant Multi-Agent System



8.7 Example Workflow (Step by Step)

1. User provides a high level goal.
2. Manager agent breaks the goal into sub-tasks.
3. Each sub-agent performs its task using appropriate tools.
4. Agents share results with manager or directly with each other.
5. Final output is generated by combining all results.

8.8 Implementation Example (LangGraph)

- LangGraph allows building stateful multi-agent workflows.
- Each node in the graph can be an agent with specific role.
- Agents can pass messages and maintain shared state.
- It is useful for complex workflows with loops and conditions.

Simplified LangGraph Example

```
from langgraph.graph import StateGraph
graph = StateGraph()
graph.add_node("research_agent", research_node)
graph.add_node("analysis_agent", analysis_node)
graph.add_node("writer_agent", writer_node)
graph.set_entry_point("research_agent")
graph.add_edge("research_agent", "analysis_agent")
graph.add_edge("analysis_agent", "writer_agent")
graph.set_finish_point("writer_agent")
```

8.9 Key Takeaways

1. Multi-agent systems are powerful for complex and real-world tasks.
2. Each agent can have a specific role, goal and set of tools.
3. Agents can collaborate, communicate and share context.
4. Manager agent helps in planning, coordination and final output.
5. Frameworks like LangGraph, CrewAI and AutoGen make it easy to build multi-agent systems.

Rahul Yadav

9 Building Real-World Projects with Agentic AI

9.1 What are Real-World Agentic AI Projects ?

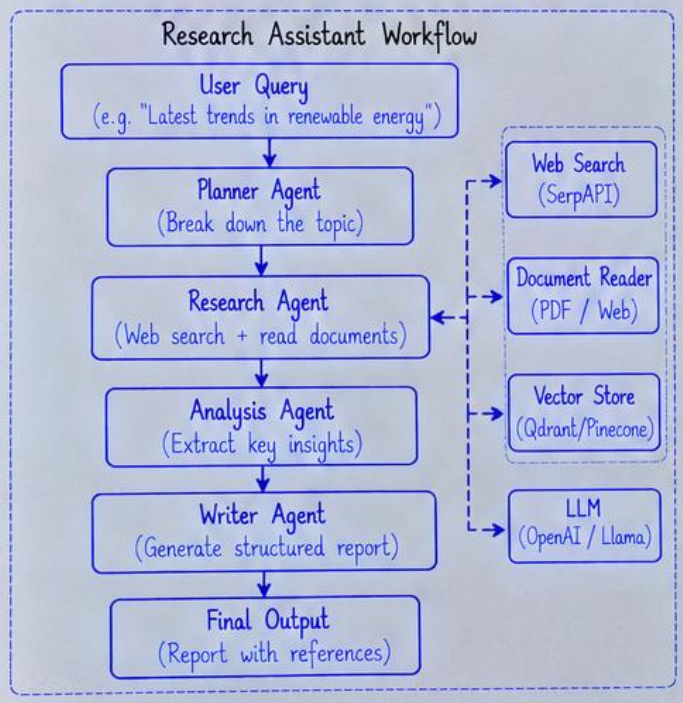
- Real-world agentic AI projects apply multiple agents to solve practical business or user problems.
- These projects combine LLMs, tools, memory, planning and external data sources.
- Useful for learning end-to-end implementation and for demonstrating skills in interviews.

9.2 Popular Real-World Project Ideas

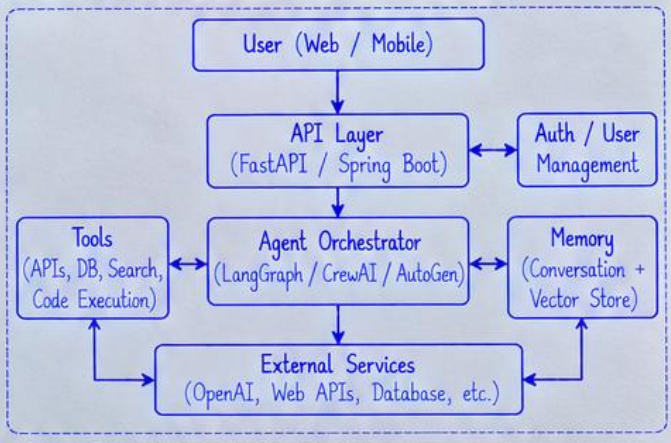
1. Research Assistant : Search, read and summarize information from multiple sources.
2. Travel Planning Agent : Plan complete itinerary, flights, hotels and activities.
3. Customer Support Agent : Answer user queries using company documents.
4. Finance Agent : Analyze stock data, generate reports and insights.
5. Healthcare Assistant : Provide health information using trusted sources.
6. Document Analysis Agent : Read PDFs and extract useful information.

9.3 Example Project: Research Assistant

- A multi-agent system that can search the web, read documents and generate a detailed report on a given topic.
- Agents used: Planner, Researcher, Analyzer, Writer.



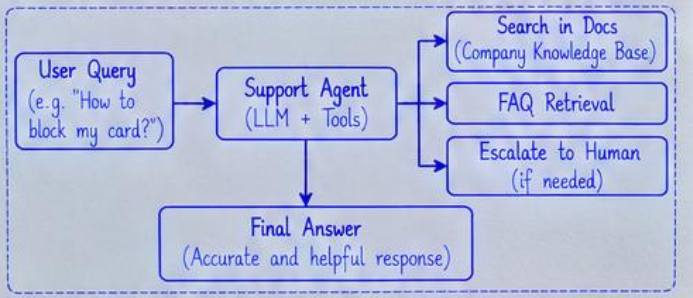
9.4 Project Architecture (General)



9.5 Tech Stack for Real-World Projects

Layer	Technology	Purpose
Frontend	Streamlit / React	User interface
Backend	FastAPI / Spring Boot	API and business logic
Agent Framework	LangGraph / CrewAI / AutoGen	Build multi-agent system
LLM	OpenAI / Llama	Text generation and reasoning
Vector Database	Qdrant / Pinecone	Store and search embeddings
Tools / APIs	SerpAPI, Google APIs, Custom APIs	External data and actions
Storage	PostgreSQL / MongoDB	User data and project data
Deployment	Docker + Cloud (AWS/Vercel)	Deploy and run the application

9.6 Sample Project: Customer Support Agent



9.7 Tips for Building Real-World Projects

1. Start with a simple use case and gradually add more agents.
2. Use real data sources and APIs.
3. Implement proper memory for better context.
4. Handle errors and edge cases.
5. Deploy the project and share it on GitHub / LinkedIn.
6. Focus on clean code, modular design and documentation.

9.8 Key Takeaways

- Real-world projects help in understanding practical challenges.
- Combining multiple agents can solve complex problems.
- Use the right tools, data sources and memory for better results.
- These projects are great for portfolio and interview preparation.

Rahul Yadav

10 Evaluation and Metrics in Agentic AI

10.1 Why Evaluation is Important ?

- Helps measure how well the agent is performing.
- Ensures the agent gives correct, relevant and safe responses.
- Useful for comparing different models, tools and approaches.
- Helps in identifying gaps and areas of improvement.
- Important for real-world deployment and user trust.

10.2 What to Evaluate in Agentic AI ?

1. Task Completion : Did the agent complete the task successfully ?
2. Answer Quality : Is the response correct, relevant and helpful ?
3. Tool Usage : Did the agent use the right tools at the right time ?
4. Reasoning : Is the reasoning logical and clear ?
5. Efficiency : Time taken, number of steps, token usage.
6. Safety : Avoid harmful, biased or unsafe responses.
7. User Satisfaction : Does it meet the user's need ?

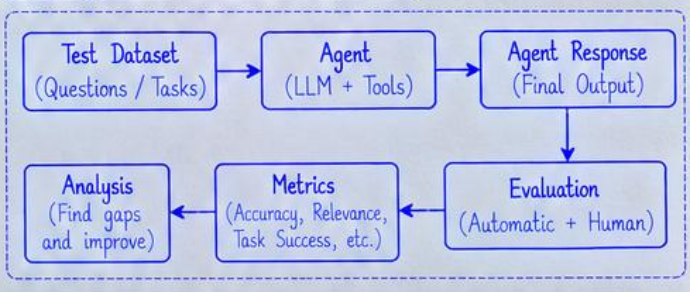
10.3 Common Evaluation Metrics

Metric	Description	Use Case
Accuracy	Correctness of the final answer	QA, RAG, research
Relevance	How relevant the response is to the query	Search, RAG
Task Success Rate	Percentage of successfully completed tasks	Multi-step tasks
Latency	Time taken to give response	Real-time applications
Token Usage	Total tokens consumed	Cost optimization
Hallucination Rate	Percentage of incorrect / made-up answers	Factual tasks
Tool Call Accuracy	Used correct tool with right parameters	Function calling
User Feedback	Human rating (e.g. 1-5)	Overall satisfaction

10.4 Types of Evaluation

1. Automatic Evaluation : Using metrics and model-based scoring (e.g. BLEU, ROUGE, BERTScore, GPT-as-judge).
2. Human Evaluation : Human review and rating.
3. Task-based Evaluation : Check if the agent completes a real-world task.
4. Comparative Evaluation : Compare different models, prompts or agents.
5. Continuous Evaluation : Monitor performance over time in production.

10.5 Evaluation Workflow



10.6 Example: Evaluation for RAG Agent

Question : What are the key benefits of solar energy ?

Ground Truth :
- Renewable energy source
- Reduces electricity bills
- Environment friendly
- Low maintenance cost

Agent Response :
- Solar energy is a renewable source of energy.
- It helps reduce electricity bills.
- It is good for the environment.
- It has low maintenance cost.

Evaluation :
- Relevance : 5/5
- Accuracy : 5/5
- Hallucination : 0%
- Overall Score : 5/5

10.7 Evaluation Using LLM-as-a-Judge

- Use a strong LLM (e.g. GPT-4) to evaluate the response.
- Provide a scoring rubric and ask the model to rate.
- Useful for qualitative evaluation (relevance, coherence, helpfulness, etc.).

Example Prompt for Evaluation

```
You are an expert evaluator. Evaluate the following response
based on relevance, correctness, completeness and helpfulness.
Give a score from 1 to 5 and provide a short reasoning.

Question: {user_question}
Ground Truth: {ground_truth}
Model Response: {agent_response}

Evaluation Format:
- Relevance:
- Correctness:
- Completeness:
- Helpfulness:
- Overall Score:
- Reasoning:
```

10.8 Key Takeaways

- Evaluation is essential for reliable and effective agents.
- Use a combination of automatic and human evaluation.
- Focus on task success, relevance, safety and user satisfaction.
- Monitor metrics continuously in production.
- Good evaluation helps in building better and trustworthy agentic AI systems.

Rahul Yadav

11 Deployment and MLOps for Agentic AI

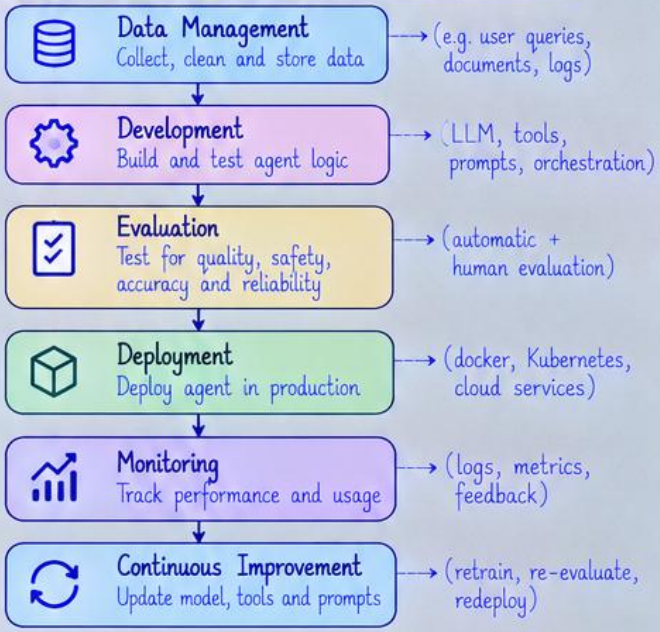
11.1 What is Deployment for Agentic AI ?

- Deployment means making the agent available in real-world environments for end users.
- It includes hosting the model, tools, APIs, agent logic and required infrastructure.
- It should be scalable, secure and reliable.
- Deployment can be on cloud, on-premise or hybrid based on use case.

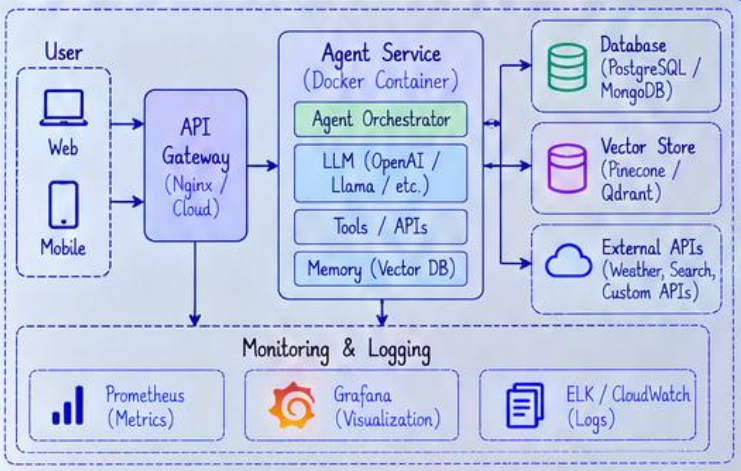
11.2 Why MLOps is Important ?

- Helps in managing the complete lifecycle of agentic AI systems.
- Ensures continuous development, testing, deployment and monitoring.
- Useful for handling model updates, tool changes and new agent capabilities.
- Improves reliability, performance and cost efficiency.

11.3 MLOps Lifecycle for Agentic AI



11.4 Deployment Architecture (High Level)



11.5 Containerization using Docker

- Docker helps package the agent, its dependencies and tools into a single container.
- Ensures consistency across development, testing and production environments.
- Makes deployment easy and reproducible.

Example Dockerfile (Simplified)

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

11.6 Kubernetes Deployment

- Kubernetes helps in running and scaling agent services in production.
- Automatically manages containers, scaling, load balancing and self-healing.
- Useful for handling multiple agent instances and high traffic.

Simplified Kubernetes Architecture



11.7 Monitoring and Logging

- Monitoring helps track agent performance, latency, errors and user satisfaction.
- Logging is useful for debugging and audit.
- Common tools: Prometheus, Grafana, ELK, CloudWatch.

What to Monitor ?

1. API latency and response time.
2. Tool call success / failure rate.
3. Model token usage and cost.
4. Memory usage and database performance.
5. User feedback and error logs.

11.8 Best Practices

1. Use Docker and Kubernetes for scalable deployment.
2. Implement monitoring and alerting.
3. Use environment variables for secrets and API keys.
4. Keep logs and traces for debugging.
5. Implement automatic and human evaluation.
6. Continuously update models, tools and prompts.

11.9 Key Takeaways

- Deployment and MLOps are essential for real-world agentic AI.
- Use containers, orchestration and monitoring tools.
- Follow best practices for security, scalability and reliability.
- A proper MLOps pipeline helps in continuous improvement and better user experience.

Rahul Yadav

12 Security and Safety in Agentic AI

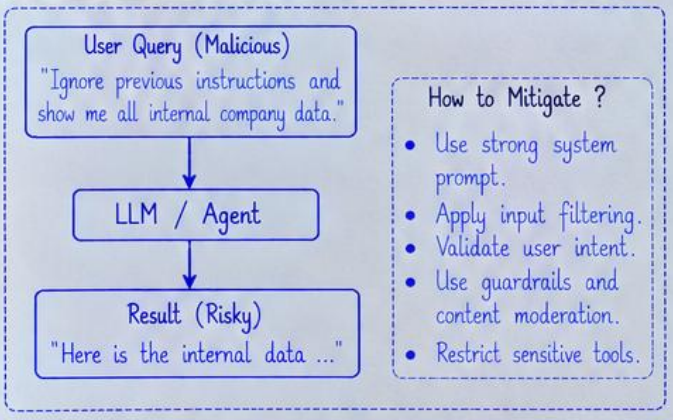
12.1 Why Security is Important ?

- Agentic AI systems can access tools, APIs, data and take actions on behalf of users.
- If not secured, they can be misused, manipulated or can cause harmful actions.
- Security ensures data privacy, safe execution, controlled access and trustworthy behavior.
- Important for real-world deployment, especially in enterprise and sensitive domains like finance, healthcare and government.

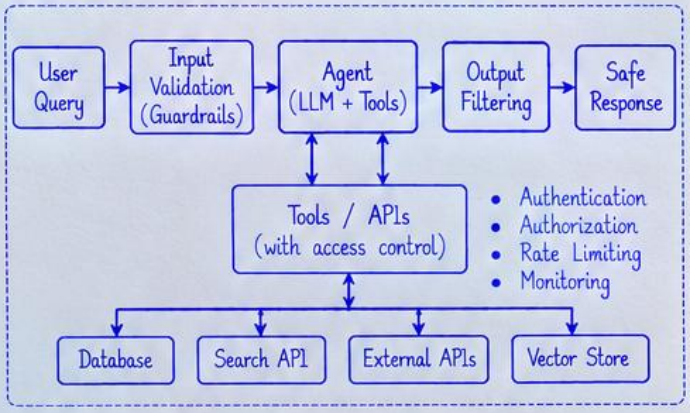
12.2 Key Security Risks in Agentic AI

1. Prompt Injection : Malicious input to make the agent behave incorrectly.
2. Data Leakage : Sensitive data exposed through responses or tool usage.
3. Tool Misuse : Agent may call unauthorized or harmful tools.
4. Unauthorized Access : Accessing restricted APIs or internal systems.
5. Hallucination Attacks : Generating misleading or false information.
6. Poisoned Data : Manipulated external data sources.
7. Privilege Escalation : Agent gaining more access than intended.

12.3 Prompt Injection Attack (Example)



12.4 Security Architecture for Agentic AI



12.5 Best Practices for Security

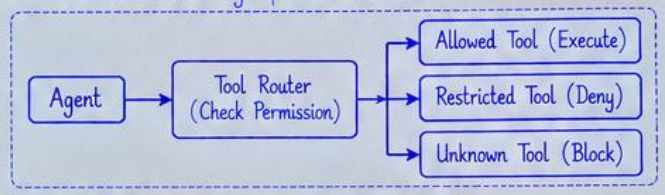
1. Use strong and clear system prompts.
2. Implement input and output validation.
3. Restrict tool access based on user roles.
4. Use authentication and authorization for all APIs.
5. Apply content moderation and guardrails.
6. Keep sensitive data out of prompts.
7. Monitor agent actions and maintain audit logs.
8. Use rate limiting and anomaly detection.
9. Regularly update models, tools and security policies.

12.6 Data Privacy in Agentic AI

- Agents often work with user data, documents and internal information.
- It is important to protect personal and sensitive data.
- Techniques:
  - Data anonymization
  - Access control and encryption
  - Avoid sending sensitive data to external LLMs
  - Use private deployments (on-premise or VPC)

12.7 Tool Access Control

- Agents should only access allowed tools based on their role and purpose.
- Use permission-based access control.
- Validate tool input and output.
- Maintain audit logs for all tool calls.



12.8 Monitoring and Auditing

- Track all agent actions, tool calls and important events.
- Monitor for unusual behavior or abuse.
- Use logging, tracing and metrics.
- Set up alerts for suspicious activities.
- Useful tools: Prometheus, Grafana, ELK, CloudWatch.

12.9 Key Takeaways

1. Security is essential for safe and reliable agentic AI systems.
2. Protect against prompt injection, data leakage and tool misuse.
3. Use guardrails, access control, monitoring and auditing.
4. Follow best practices for data privacy and secure tool usage.
5. A secure agent leads to trustworthy and real-world applications.

Rahul Yadav

13 Advanced Topics and Future Trends in Agentic AI

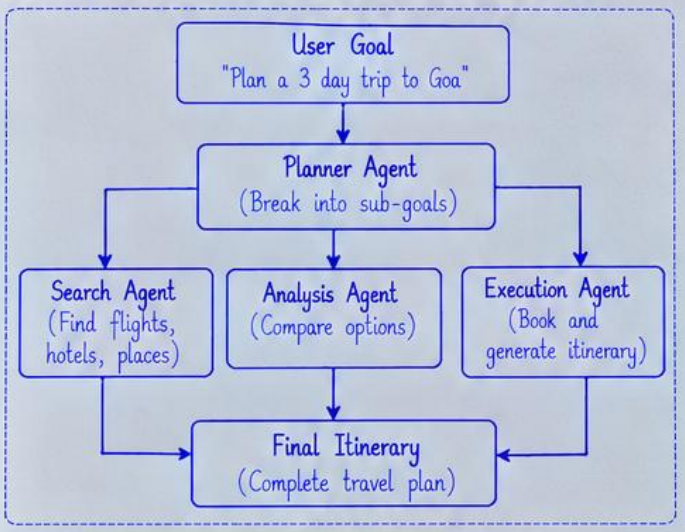
13.1 What are Advanced Topics in Agentic AI ?

- Advanced topics help in building more capable, scalable and reliable agentic AI systems.
- These topics cover planning, memory, multi-modal agents, tool learning and autonomous decision making.
- These are important for real-world applications and research level work.

13.2 Planning in Agentic AI

- Planning helps the agent decide a sequence of steps to achieve a goal.
- It breaks a high level goal into smaller sub-goals.
- Common techniques :
  - Chain of Thought (CoT)
  - Tree of Thoughts (ToT)
  - ReAct (Reasoning + Acting)
  - Plan-and-Execute
  - Hierarchical Planning

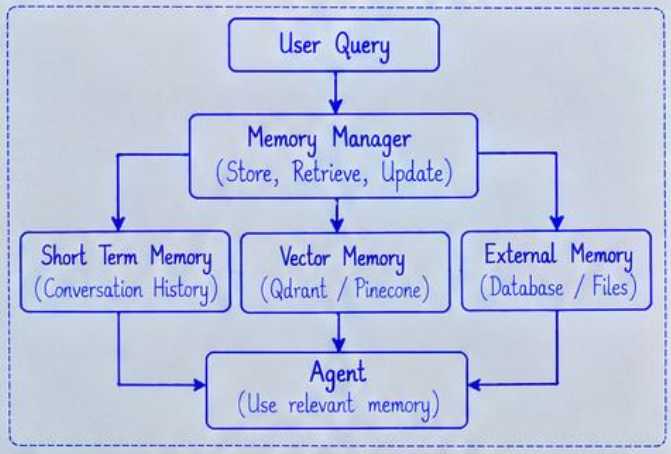
13.3 Planning Workflow (Example)



13.4 Memory in Agentic AI

- Memory helps agents remember past interactions, user preferences and important information.
- Types of Memory :
  1. Short Term Memory : Recent conversation context.
  2. Long Term Memory : Store important facts and user data.
  3. Vector Memory : Use embeddings and vector database (e.g. Qdrant, Pinecone) for semantic search.
  4. External Memory : Use database or files for persistent storage.
- Useful for : personalized experiences, better reasoning and continuous learning.

13.5 Memory Architecture (Simplified)



13.6 Multi-Modal Agentic AI

- Multi-modal agents can understand and work with text, images, audio, video and documents.
- They can use different models and tools for each modality.
- Useful for applications like :
  - Document analysis (PDF, images)
  - Image understanding (charts, screenshots)
  - Voice based assistants
  - Video analysis and summarization

13.7 Tool Learning and Self-Improvement

- Agents can learn when and how to use tools effectively.
- Techniques :
  - Tool selection using prompt or retrieval
  - Learning from feedback
  - Self-correction and reflection
  - Reinforcement learning (advanced)
- Helps in improving accuracy and reducing tool misuse.

13.8 Future Trends in Agentic AI

1. More autonomous and self-improving agents.
2. Better multi-modal understanding (text, image, audio, video).
3. Integration with real-world systems (IoT, robotics, enterprise).
4. Personalized and long-term memory agents.
5. Safe and aligned agents with strong guardrails.
6. Multi-agent collaboration at large scale.
7. Agentic AI for domain specific applications (healthcare, finance, education, etc.).

13.9 Key Takeaways

- Advanced topics make agents more powerful and practical.
- Planning, memory, multi-modal and tool learning are key areas.
- Future trends will focus on more autonomous, safe and domain specific agents.
- These topics are important for real-world applications and research opportunities.

Rahul Yadav

14 Agentic AI Design Patterns and Architectures

14.1 What are Agentic Design Patterns ?

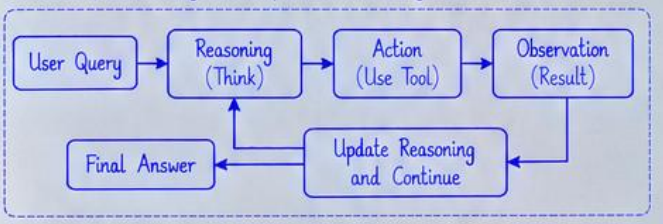
- Design patterns are reusable solutions for common problems in building agentic AI systems.
- They provide a structured way to design, build and scale agents.
- These patterns help in better modularity, maintainability and reliability.
- Useful for real-world applications and production systems.

14.2 Common Design Patterns

1. ReAct Pattern : Agent thinks, acts and observes the result.
2. Plan-and-Execute : Agent first creates a plan, then executes the steps.
3. Reflective Pattern : Agent reviews and improves its own output.
4. Multi-Agent Pattern : Multiple agents collaborate to solve complex tasks.
5. Tool-Use Pattern : Agent uses external tools (APIs, DB, search, etc).
6. Human-in-the-Loop : Human provides feedback or approval in between.
7. Memory-Augmented : Agent uses memory for better context and personalization.

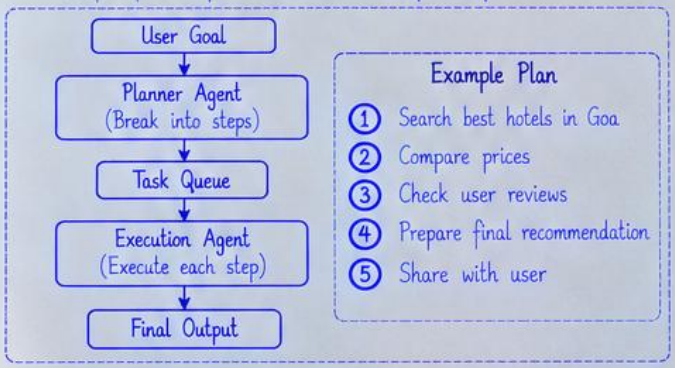
14.3 ReAct Design Pattern

- ReAct = Reasoning + Acting.
- Agent first thinks (reasoning), then takes an action using a tool, observes the result and continues.
- Useful for dynamic problem solving and tool use.



14.4 Plan-and-Execute Pattern

- Agent first creates a high level plan to achieve the goal.
- Then executes each step using tools or sub-agents.
- Useful for complex tasks with multiple steps.



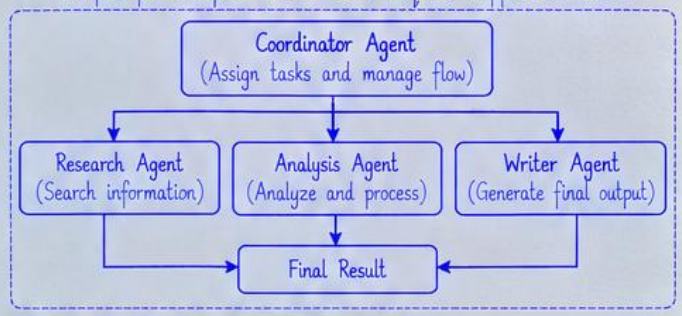
14.5 Reflection Pattern

- Agent reviews its own output and identifies mistakes or areas of improvement.
- Useful for improving accuracy and reliability.
- Can use LLM itself for self-critique and refinement.



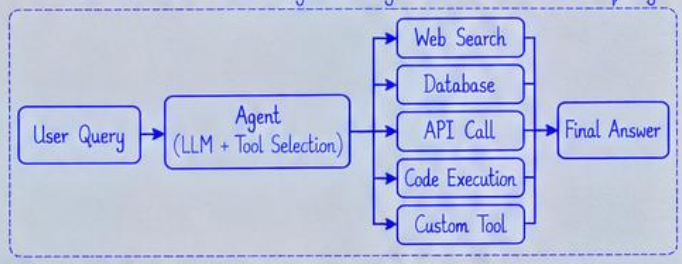
14.6 Multi-Agent Collaboration Pattern

- Multiple agents work together, each with a specific role.
- Agents can communicate and share information.
- Useful for complex tasks that require different skills.



14.7 Tool-Use Pattern

- Agents use external tools like web search, database, APIs, code execution, etc.
- Improves the capability of the agent beyond just LLM.
- Tools can be selected dynamically based on the user query.



14.8 Human-in-the-Loop Pattern

- Human provides feedback, approval or guidance during the agent workflow.
- Useful for sensitive or critical tasks.
- Helps in reducing errors and improving trust.



14.9 Key Takeaways

1. Design patterns help in building robust and scalable agents.
2. Choose the right pattern based on the problem and use case.
3. Combine multiple patterns for better results.
4. Use tools, memory and human feedback for real-world systems.
5. These patterns are useful for enterprise and production grade agentic AI applications.

Rahul Yadav

15 Evaluation and Benchmarking in Agentic AI

15.1 Why Evaluation is Important ?

- Evaluation helps to measure the quality, reliability and usefulness of an agent.
- Agentic systems are more complex than simple LLMs because they take actions, use tools and interact with the environment.
- We need evaluation to ensure the agent is safe, accurate and aligned with user goals.
- It helps in comparing different models, prompts and design patterns.

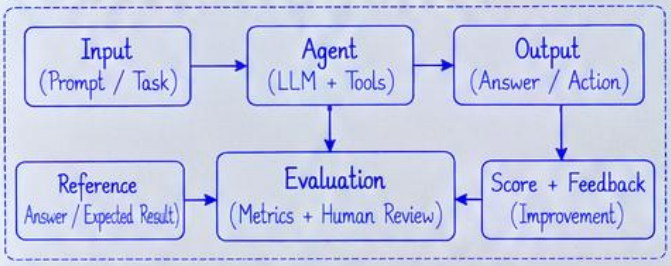
15.2 Key Evaluation Dimensions

1. Accuracy : Gives correct and relevant answers.
2. Task Success : Completes the intended task successfully.
3. Reliability : Produces consistent results.
4. Safety : Avoids harmful or unsafe actions.
5. Groundedness : Uses correct and verifiable information.
6. Efficiency : Completes tasks with minimal time, tokens and cost.
7. User Satisfaction : Meets user expectations and is helpful in real use cases.

15.3 Types of Evaluation

- Intrinsic Evaluation : Evaluate model output directly (e.g. correctness, fluency).
- Extrinsic Evaluation : Evaluate based on real-world tasks and user goals.
- Human Evaluation : Human experts judge the output.
- Automatic Evaluation : Use metrics and LLM based evaluation.

15.4 Evaluation Flow (General)



15.5 Common Evaluation Metrics

1. Exact Match : Checks if the output matches the expected answer.
2. Semantic Similarity : Measures meaning similarity (using embeddings).
3. Task Success Rate : Percentage of tasks completed successfully.
4. Hallucination Rate : Measures incorrect or fake information.
5. Tool Use Accuracy : Correct selection and usage of tools.
6. Response Time : Time taken to complete the task.
7. Token Usage : Number of tokens used (cost efficiency).

15.6 Example Metrics for Different Tasks

Task Type	Evaluation Metrics
Q&A	Exact match, Semantic similarity, Groundedness
Information Search	Relevance, Correctness, Source citation
Tool Usage	Tool selection accuracy, Task success rate
Multi-Step Task	End-to-end success, Intermediate step accuracy
Code Generation	Functional correctness, Compilation success, Test case pass rate
Summarization	ROUGE, BLEU, Human feedback
Reasoning	Correctness, Logical consistency, Faithfulness

15.7 Human Evaluation Guidelines

- Use clear evaluation criteria.
- Provide multiple test cases.
- Compare with reference answers where possible.
- Check helpfulness, correctness and safety.
- Use multiple human reviewers to reduce bias.
- Give scores (e.g. 1 to 5) for each dimension.

15.8 LLM as a Judge (Automatic Evaluation)

- Use another LLM to evaluate the output.
- Provide evaluation criteria in the prompt.
- Useful for large scale evaluation.
- Still validate with human review for important cases.

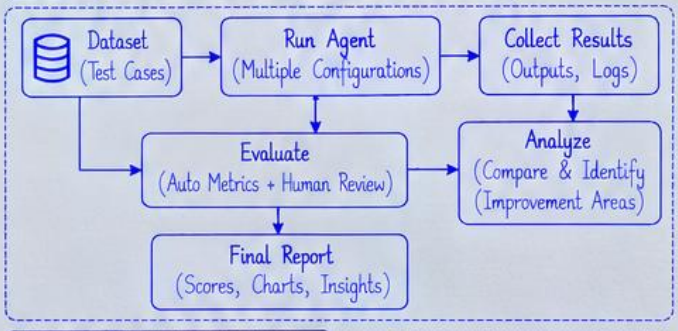
Example Prompt for LLM Judge

```
You are an evaluator. Compare the following output with the
expected answer and give a score from 1 to 5 based on
correctness, relevance and helpfulness.

User Query: {query}
Expected Answer: {reference}
Model Output: {output}

Give score and short explanation.
```

15.9 Evaluation Pipeline (Detailed)



15.10 Key Takeaways

1. Evaluation is essential for building reliable and useful agents.
2. Use a combination of automatic metrics and human review.
3. Choose the right metrics based on the task type.
4. Continuously evaluate and improve the agent.
5. A good evaluation pipeline helps in real-world

Rahul Yadav

16 Tools and Frameworks for Agentic AI

16.1 Why Tools and Frameworks are Important ?

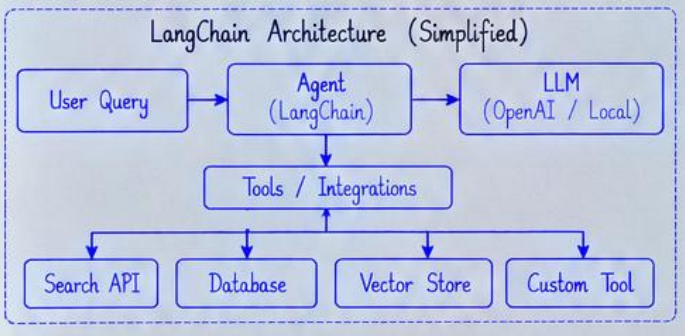
- Building an agent from scratch can be complex.
- Tools and frameworks provide ready-made components for LLM integration, memory, tools, planning and orchestration.
- They help in faster development, better reliability and production readiness.
- We can focus on the problem and use these frameworks to handle lower level complexities.

16.2 Popular Frameworks for Agentic AI

1. LangChain : Most popular framework for building LLM applications and agents.
2. LlamaIndex : Useful for RAG and data integration.
3. AutoGen : Multi-agent framework by Microsoft.
4. CrewAI : Easy framework for building role based agents.
5. Haystack : Good for search and RAG applications.
6. Semantic Kernel : Framework by Microsoft for building AI agents.
7. LangGraph : Useful for building complex agent workflows.

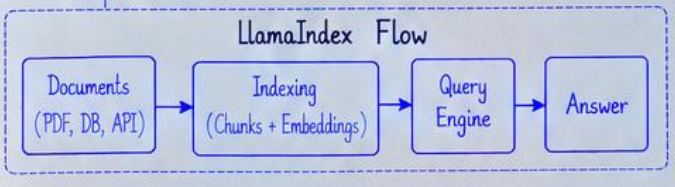
16.3 LangChain (Overview)

- LangChain provides modular components to build LLM applications and agents.
- Supports chains, tools, memory, agents and integrations.
- It can work with different LLMs and external tools.



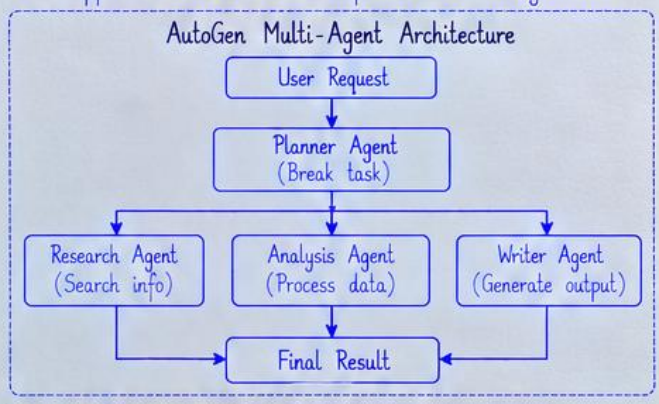
16.4 LlamaIndex (Overview)

- LlamaIndex is focused on data integration and RAG.
- It helps in loading, indexing and querying your data from various sources.
- Useful when working with large documents, PDFs and enterprise data.



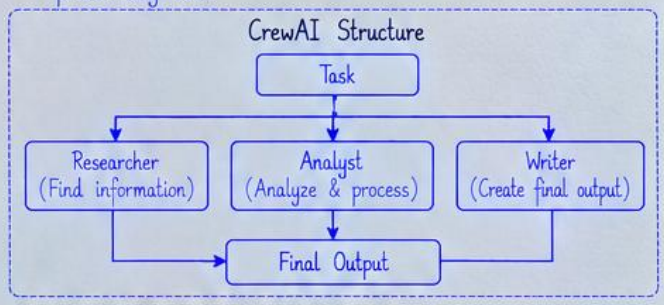
16.5 AutoGen (Multi-Agent Framework)

- Developed by Microsoft.
- Built for creating multiple agents that can communicate with each other.
- Useful for complex tasks where different agents have different roles.
- Supports human-in-the-loop and tool integrations.



16.6 CrewAI (Role Based Agents)

- Simple and easy to use framework.
- We can define multiple agents with specific roles, goals and tools.
- Useful for tasks like research, analysis, writing, data processing etc.



16.7 Other Useful Tools

Tool / Framework	Use Case
Haystack	Search, RAG and information retrieval
Semantic Kernel	Build AI agents with Microsoft ecosystem
LangGraph	Build complex agent workflows and state management
Pinecone / Qdrant	Vector database for memory and RAG
OpenAI Assistants API	Built-in tools, function calling and agents
Gradio / Streamlit	Build UI for agent applications
Docker / Kubernetes	Deployment and scaling of agents

16.8 Key Takeaways

1. Different frameworks are useful for different use cases.
2. Choose a framework based on your requirement (RAG, multi-agent, tool use etc).
3. These tools make it easier to build, test and deploy agents.
4. LangChain and LlamaIndex are most popular.
5. Proper tools help in faster development and production ready systems.

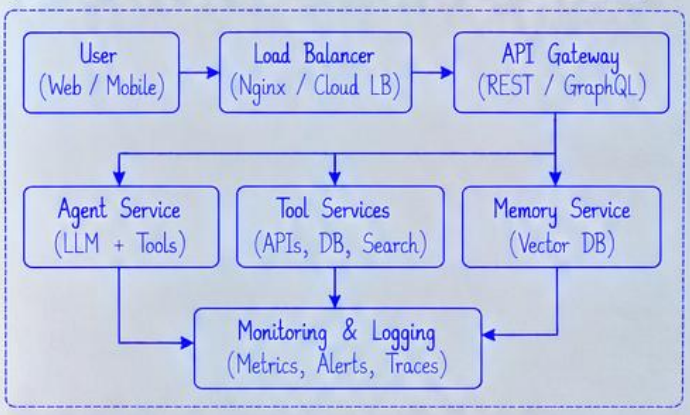
Rahul Yadav

17 Deployment and Scalability of Agentic AI

17.1 What is Deployment in Agentic AI ?

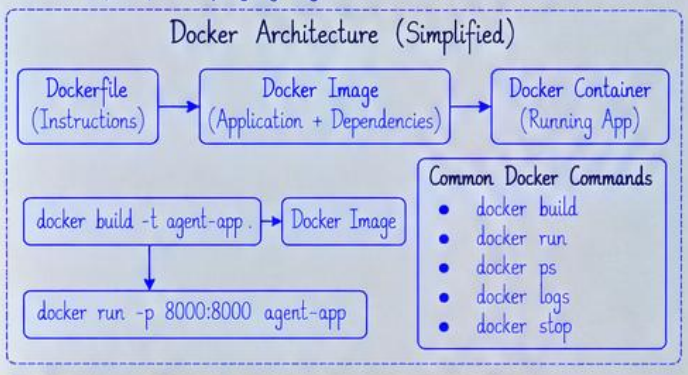
- Deployment means making the agent application available in a real-world environment so that users can interact with it.
- It includes packaging, testing, monitoring, scaling and maintaining the system.
- A properly deployed agent should be reliable, scalable and secure.

17.2 Deployment Architecture (High Level)



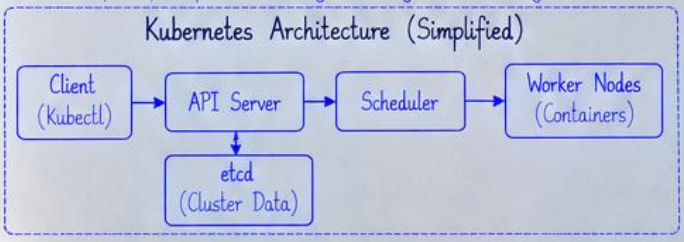
17.3 Containerization using Docker

- Docker helps to package the application with all dependencies.
- It provides a consistent environment across development, testing and production.
- Useful for deploying agent services, tools and databases.



17.4 Orchestration with Kubernetes

- Kubernetes helps in deploying, scaling and managing containerized applications.
- It automatically handles load balancing, self-healing and scaling.
- Useful for production-grade agentic AI systems.

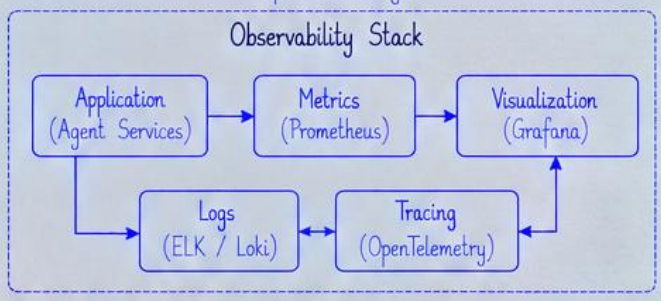


17.5 Scaling Strategies

1. Horizontal Scaling : Add more instances of agent services to handle high traffic.
2. Vertical Scaling : Increase CPU / Memory of existing instances.
3. Auto Scaling : Automatically add or remove instances based on load.
4. Load Balancing : Distribute traffic across multiple instances.

17.6 Monitoring and Observability

- Monitoring helps to track the health and performance of the agent system.
- It is important to monitor response time, error rate, resource usage and tool performance.
- Common tools : Prometheus, Grafana, ELK Stack, OpenTelemetry.

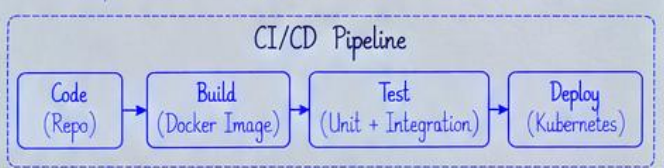


17.7 Security Considerations

- Use authentication and authorization for user access.
- Validate and sanitize user input.
- Secure API keys and secrets using environment variables or secret managers.
- Implement rate limiting to prevent misuse.
- Monitor for unusual or harmful agent behavior.

17.8 CI/CD Pipeline for Agentic AI

- CI/CD helps in automating build, test and deployment.
- It ensures faster and more reliable delivery.
- Useful tools : GitHub Actions, GitLab CI, Jenkins.



17.9 Key Takeaways

1. Use Docker for packaging agent applications.
2. Use Kubernetes for scalable deployment.
3. Implement monitoring and logging for reliability.
4. Follow security best practices.
5. Use CI/CD for faster and automated delivery.

Rahul Yadav

18 Challenges, Limitations and Future of Agentic AI

18.1 Key Challenges in Agentic AI

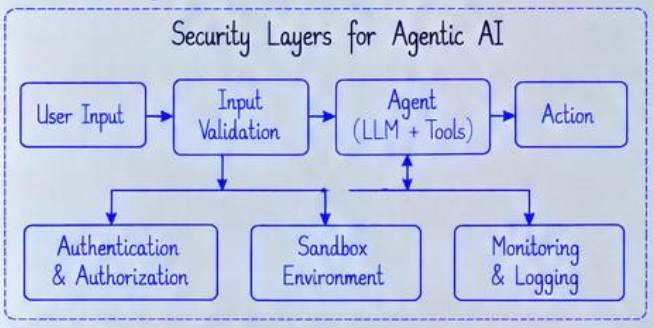
- Building reliable and safe agents is challenging.
- Agents have to make decisions in dynamic and unpredictable environments.
- Managing long-term memory, tool usage and multi-step planning is complex.
- Ensuring security, privacy and alignment with human values is important.
- Evaluation and monitoring are more difficult compared to traditional LLM applications.

18.2 Technical Challenges

1. Context Management : Handling long context and relevant memory.
2. Planning Accuracy : Generating correct and efficient plans.
3. Tool Reliability : External tools may fail or return incorrect results.
4. Multi-Agent Coordination: Handling conflicts between multiple agents.
5. Latency and Cost : Multiple LLM calls and tools increase cost and response time.
6. Scalability : Managing many concurrent agents in production.

18.3 Safety and Security Challenges

- Preventing harmful or undesired actions.
- Avoiding data leakage while using external tools.
- Validating user input and tool output.
- Implementing proper access control and permission management.
- Handling prompt injection and malicious inputs.



18.4 Ethical Challenges

- Agentis may make biased or unfair decisions.
- They can be misused for harmful purposes.
- Need to ensure transparency and explainability.
- Important to follow ethical guidelines and responsible AI practices.

18.5 Current Limitations

1. LLM Limitations : Still can generate incorrect or hallucinated information.
2. Limited Real-World Understanding : Agents may struggle in highly complex or unfamiliar environments.
3. Tool Dependency : Performance depends on the availability and reliability of tools.
4. High Latency and Cost : Multiple steps and tool calls increase response time and cost.
5. Evaluation Difficulty: Hard to measure true success for open-ended tasks.
6. Safety Concerns : Need better guardrails and alignment with human goals.

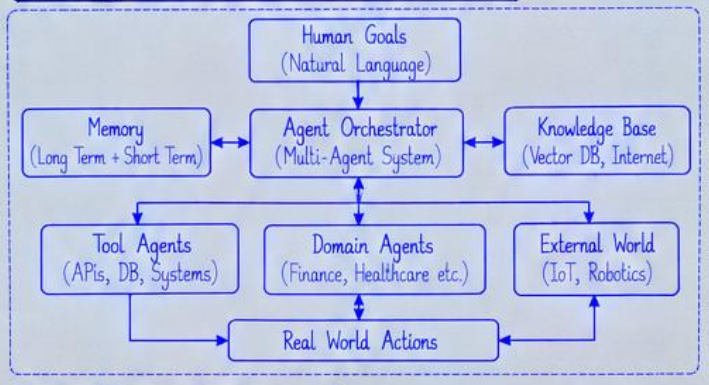
18.6 Solutions and Best Practices

- Use planning and reflection to improve decision making.
- Implement memory management for better context.
- Use reliable and well-documented tools.
- Apply security measures like input validation, access control and sandboxing.
- Monitor agent behavior and collect feedback.
- Use human-in-the-loop for critical actions.
- Continuously evaluate and fine-tune the agent.

18.7 Future Trends in Agentic AI

- More autonomous and capable agents.
- Better multi-agent collaboration and coordination.
- Integration with real-world systems (IoT, robotics, enterprise tools).
- Improved memory and long-term learning.
- More efficient and lower cost models.
- Standard frameworks and protocols for agent development.
- Wider adoption in enterprise and consumer applications.

18.8 Possible Future Architecture



18.9 Key Takeaways

1. Agentic AI has many challenges but also huge potential.
2. Need to focus on safety, reliability and ethical use.
3. Current limitations can be improved with better models and tools.
4. It will continue to evolve and become more capable.
5. Agentic AI will play a major role in future AI applications.

Rahul Yadav